



Model Order Reduction
and
Machine Learning
a.a. 2024-2025

MOR methods for nonlinear Elliptic problem

September 12, 2025

Maddalena Ghiotti s332834
Valerio Taralli s332689
Gaia Saroglia s332687

Problem Definition

- PDE:

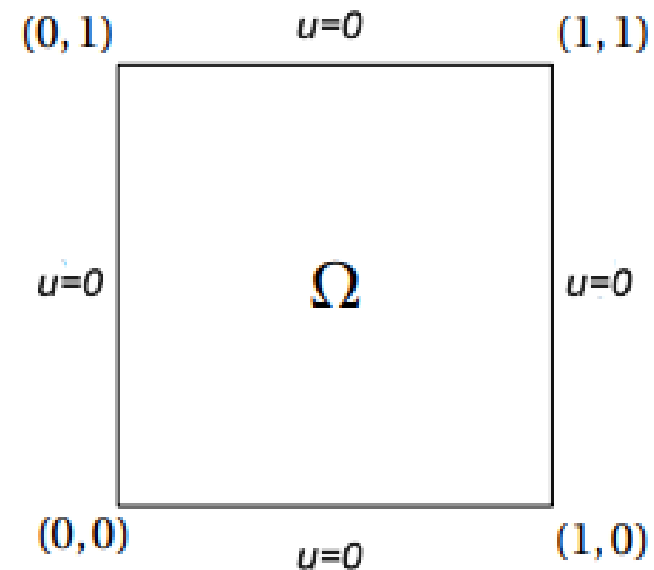
$$-\Delta u^\mu + \frac{\mu_0}{\mu_1} e^{\mu_1 u} = g^\mu + \frac{\mu_0}{\mu_1}$$

- Two cases for source term:

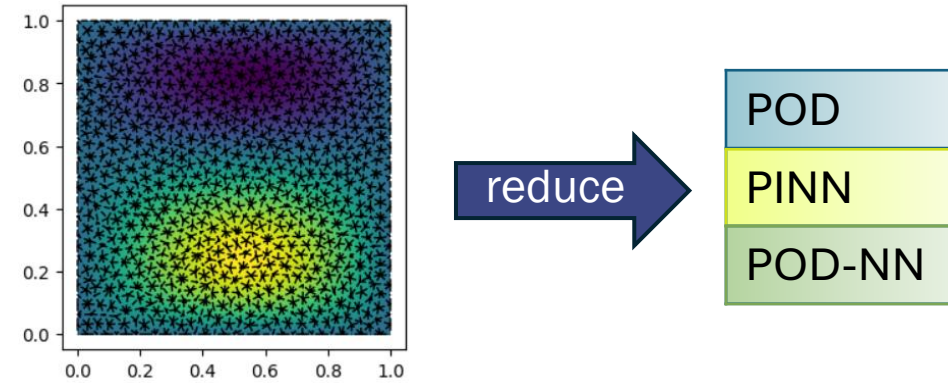
1. $g_1(\mathbf{x}) \equiv 100 \sin(2\pi x_0) \cos(2\pi x_1)$

2. $g_2^{\mu_0}(\mathbf{x}) \equiv 100 \sin(2\pi \mu_0 x_0) \cos(2\mu_0 \pi x_1)$

- Domain: $\Omega = (0,1)^2$
- Boundary conditions: homogeneous Dirichlet



Motivation & Goal



Motivation:

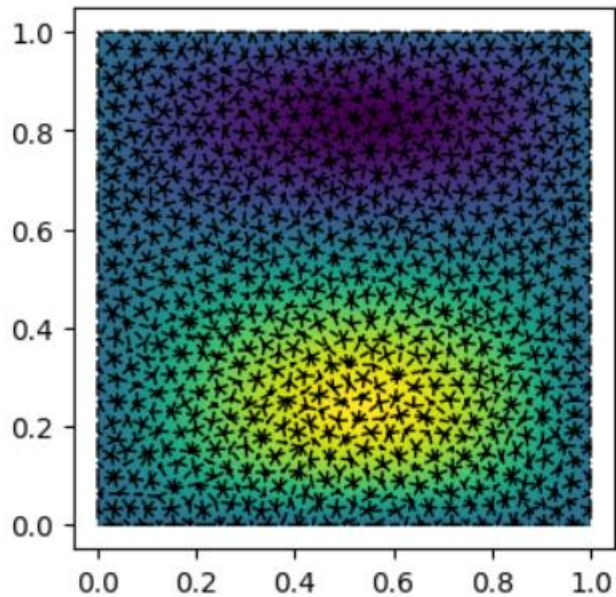
- **FOM** (FEM) is accurate but expensive
- Need Model Order Reduction (**MOR**) to reduce costs

Goal:

Compare **POD-Galerkin**, **PINN**, **POD-NN** vs **FOM** in terms of

- *Accuracy* (absolute & relative errors)
- *Computational efficiency* (speed-up)

Full Order Model (FEM)



- **Finite Elements** : discretization of weak formulation

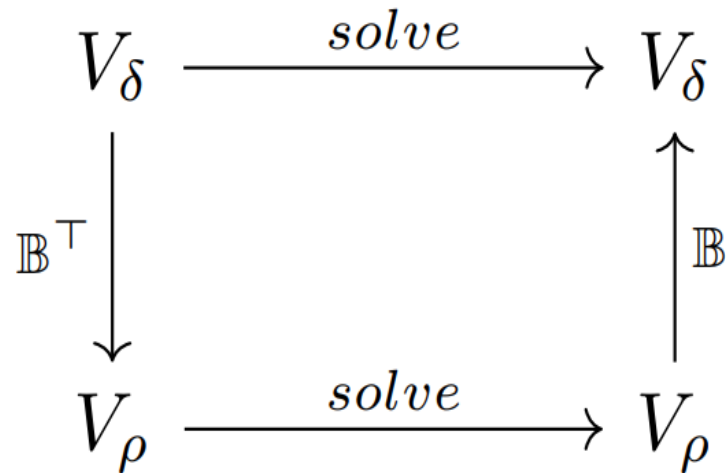
$$V_\delta = \text{span} \{ \varphi_k \}_{k=1}^{N_\delta} \subseteq V$$
$$N_\delta \equiv \dim \{ V_\delta \}_{\delta > 0}$$

- **Non linearity**: Newton iterations to obtain a matrix formulation

$$J_{r,n}^\mu [\delta u_n^\mu, v_\delta] \delta u_n^\mu = -r^\mu(u_{\delta,n}^\mu, v_\delta) \quad \forall v_\delta \in V_\delta$$

$$u_{\delta,n+1}^\mu = u_{\delta,n}^\mu + \delta u_n^\mu$$

Proper Orthogonal Decomposition (POD)



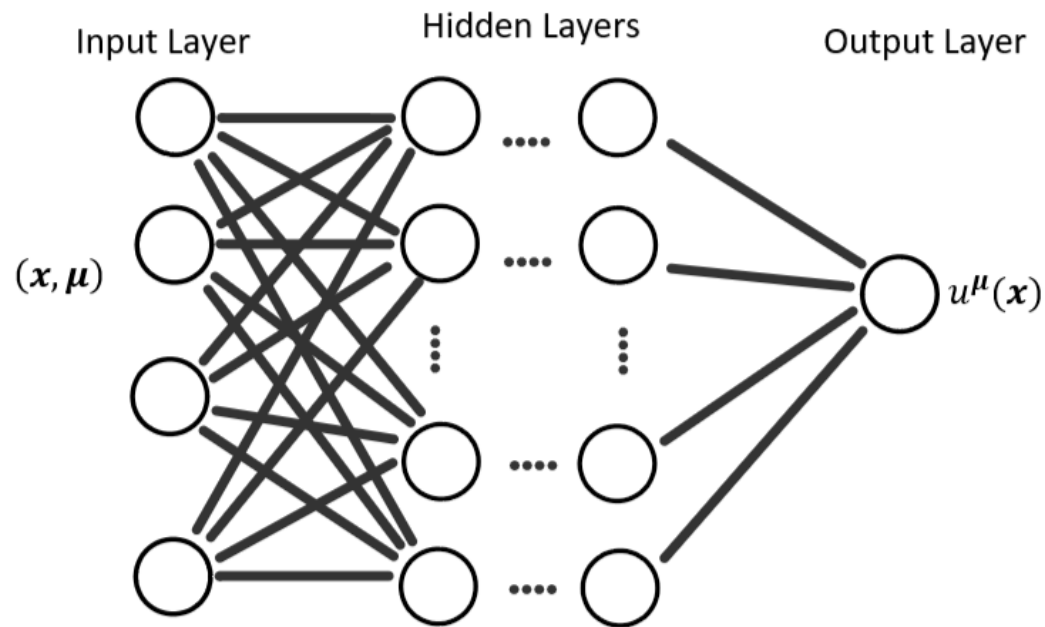
- Builds a **reduced basis** from snapshots

$$V_{\rho} = \text{span}\{\xi_k\}_{k=1}^{N_{\rho}} \subseteq V_{\delta}$$

$$\dim(V_{\rho}) = N_{\rho} \ll N_{\delta}$$

- Solve reduced system in V_{ρ} and **reconstruct solution**

Physics-Informed Neural Network (PINN)



- Train a *Neural Network*, mesh-free approach:

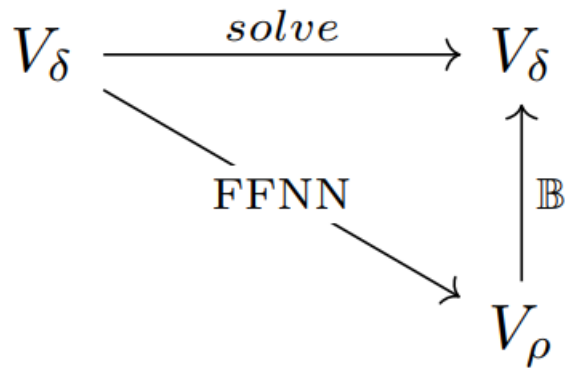
$$\mathcal{F}_{NN}: \Omega \times \mathcal{P} \rightarrow \mathbb{R}$$

$$(\mathbf{x}, \mu) \mapsto \overline{u^\mu}(\mathbf{x}) \approx u^\mu(\mathbf{x})$$

- Minimization of the loss function:

$$MSE = MSE_b^\mu + \lambda MSE_p^\mu$$

Hybrid Approach (POD-NN)



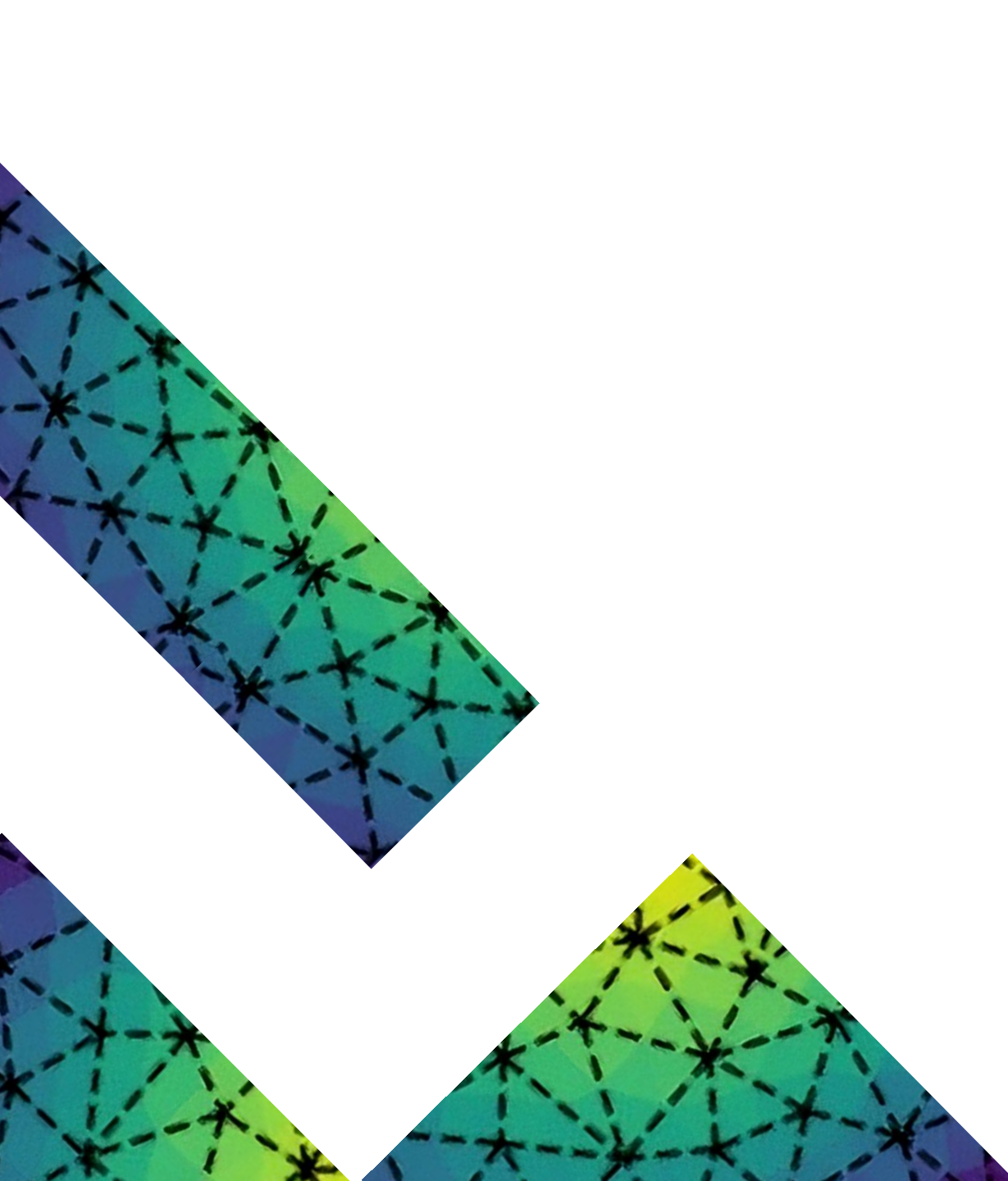
- **POD stage:** compute *snapshots* and *basis matrix*
- **NN stage:**

$$\mathcal{F}_{PODNN}: \mathcal{P} \rightarrow \mathbb{R}^{N_\rho}$$

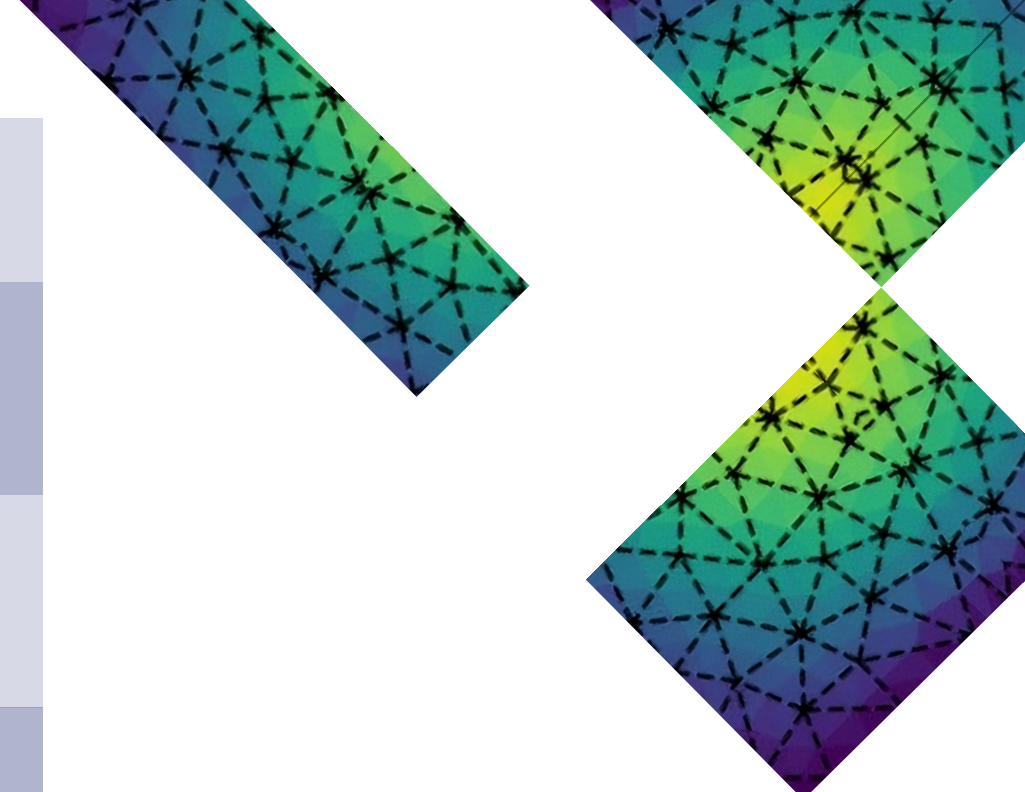
$$\mu^* \mapsto \widetilde{\mathbf{u}}^{\mu^*}$$

- **Reconstruct** the solution
- **Minimization** of the loss function:

$$\text{MSE} = \frac{1}{M} \sum_{m=1}^M ||\widetilde{\mathbf{u}}^{\mu_m} - \widehat{\mathbf{u}}^{\mu_m} ||_2^2$$

The bottom-left corner of the slide features three overlapping triangular regions. Each region contains a pattern of small, dark, star-like shapes connected by dashed lines, creating a mesh-like appearance. The colors of these regions transition from dark blue/purple on the left to bright yellow/green on the right.

Numerical Studies



FOM	FEM: P2 elements, $a_{\max} = 0.0001$ Newton iterations ($tol = 10^{-6}$, $maxI = 100$),
POD	100 snapshots, up to 10 modes s.t. energy threshold $1 - 10^{-7}$
PINN	FCNN, 5×50 neurons, tanh, 10k internal + 10k boundary points, Adam $lr = 10^{-3}$, 500k epochs
POD-NN	POD basis (1000 snapshots), FCNN 5×50 , tanh, training set 1000 samples, Adam $lr = 10^{-4}$, 500k epochs

Metrics:

- *absolute/relative* error
- speed-up

Experimental Setup

$$g_1(\mathbf{x}) \equiv 100 \sin(2\pi x_0) \cos(2\pi x_1)$$

- **POD:** *best accuracy, negligible speed-up*
- **PINN:** *very fast (after training), accuracy ~1 order worse*
- **POD-NN:** *best compromise, POD-like accuracy and strong speed-up*

<i>Method</i>	Avg Relative Error ($\times 10^{-3}$)	Avg Absolute Error ($\times 10^{-2}$)	Avg Speed-Up
POD	0.771	0.355	1.190
PINN	8.443	3.895	70.327
POD-NN	0.820	0.378	216.336

Results – Case 1

$$g_2^{\mu_0}(\mathbf{x}) \equiv 100 \sin(2\pi\mu_0 x_0) \cos(2\mu_0\pi x_1)$$

Higher complexity amplifies performance differences:

- **POD**: keeps *highest accuracy*, but *no speed gain*
- **PINN**: *strong speed-up*, but *accuracy worsens* significantly
- **POD-NN**: still *best trade-off*

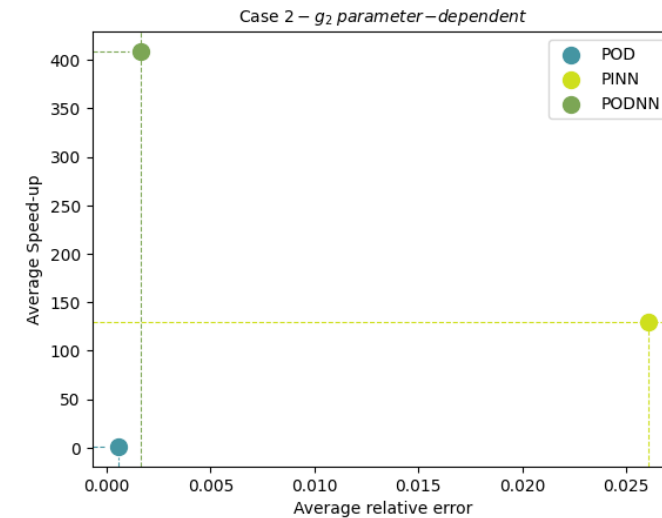
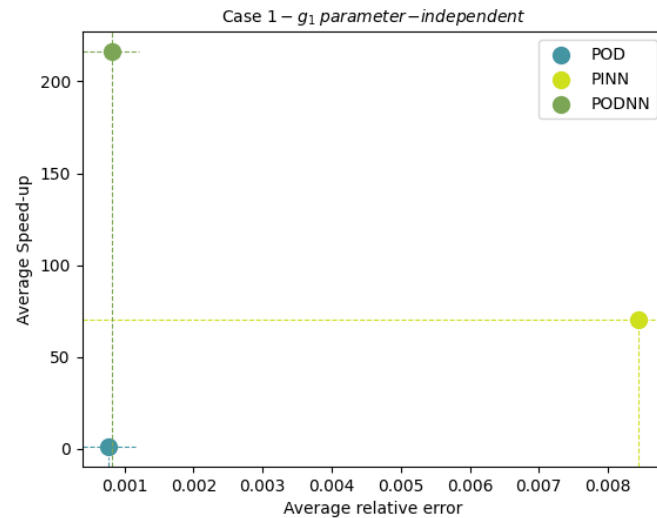
POD: 9 modes
(vs. 2 modes)

<i>Method</i>	Avg Relative Error ($\times 10^{-3}$)	Avg Absolute Error ($\times 10^{-2}$)	Avg Speed-Up
POD	0.584	0.370	1.135
PINN	26.064	16.131	129.073
POD-NN	1.621	0.966	408.778

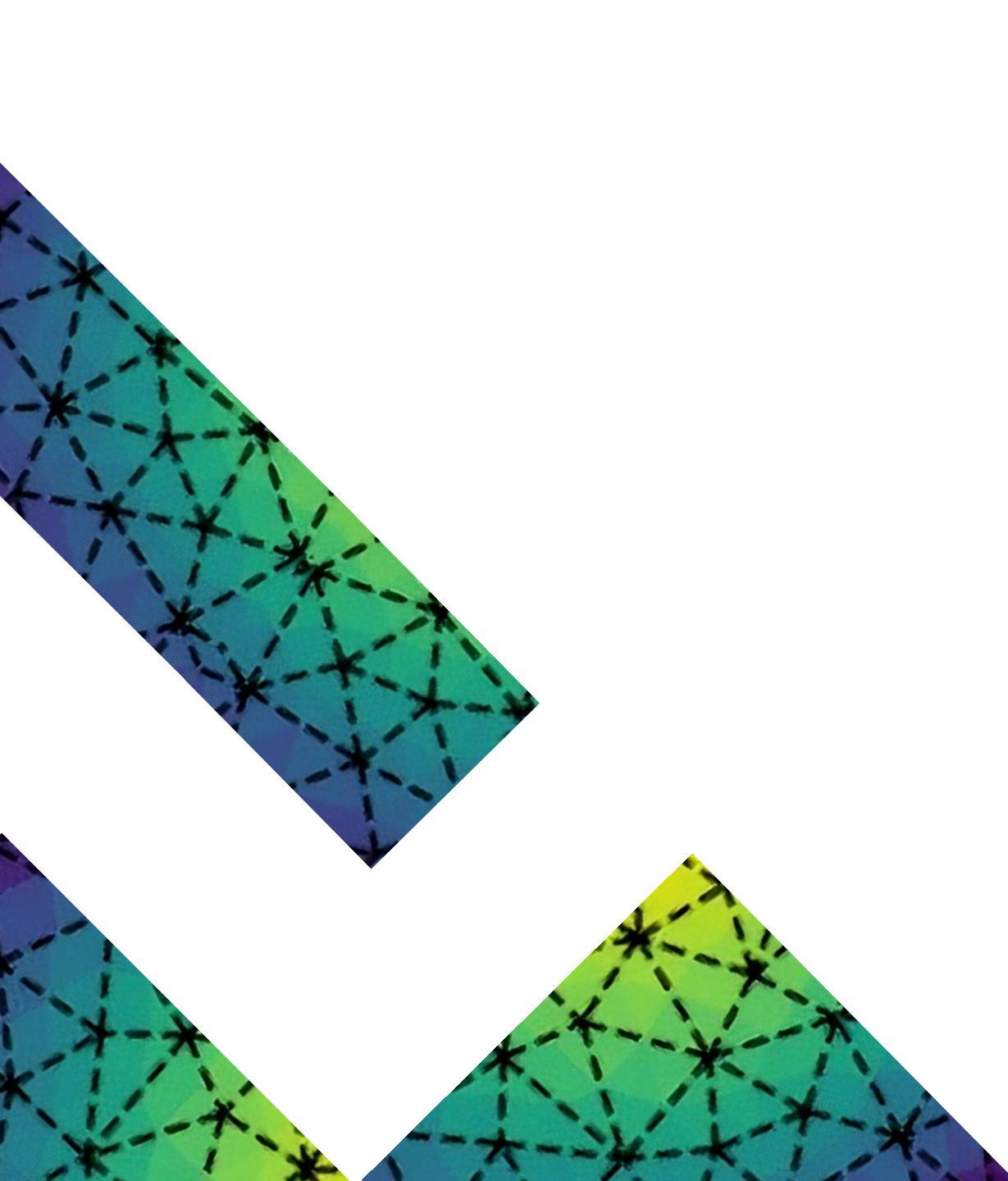
Results – Case 2

Conclusions

- POD remains the preferred method when accuracy is the primary goal;
- PINN and POD-NN offer remarkable advantages in computational speed.



Overall : **POD-NN** emerges as the most promising method when both accuracy and efficiency are required.

Three overlapping triangular shapes in the bottom-left corner, each filled with a colorful gradient (purple, blue, green, yellow) and a pattern of small black stars connected by dashed lines.

**Thank you for
listening**
